

Community Health Survey System

DevOps Mini-Project

Week 7 – Jenkins Installation and Continuous Integration

Repository: https://github.com/jyolicious/DevOps_miniProject

Jenkins Job: Community-Health-Survey-CI

Jenkins Version: 2.568.3 | **Java:** 17

2. Objective

The objective of Week 7 is to establish a Continuous Integration (CI) pipeline for the Community Health Survey System using Jenkins. This involves installing and configuring a local Jenkins server, connecting it to the project's GitHub repository, and creating a build job that compiles, tests and packages the Spring Boot application using Maven whenever the repository changes. The build job is also configured to archive the resulting JAR artifact, so that a deployable build output is produced and retained automatically as part of every successful CI run.

This deliverable demonstrates that source control changes can be automatically detected and built without manual intervention, fulfilling the core CI requirement for the project's DevOps pipeline.

3. Tools and Technologies

Tool / Technology	Role in Week 7
Jenkins 2.568.3	Continuous Integration server; builds, tests, packages and archives the application artifact.
Git / GitHub	Version control and source repository (jyolicious/DevOps_miniProject) polled by Jenkins.
Apache Maven (via mvnw)	Build automation tool used to compile, test and package the Spring Boot application.
Java 17	Runtime and compilation target for the Spring Boot application and the Jenkins build environment.
Spring Boot	Application framework for the Community Health Survey System.
Windows environment	Host operating system on which Jenkins and the build tool-chain are configured and executed.

4. Jenkins Installation and Configuration

Jenkins was installed and configured to run locally on the development machine, accessible at <http://localhost:8082>. The installed Jenkins version is 2.568.3, and the Jenkins environment was configured to use Java 17, matching the Java version required by the Community Health Survey Spring Boot application.

Global tool configuration was carried out under Manage Jenkins → Tools, where the Git executable (git.exe) was registered as the default Git installation used by Jenkins jobs for source code checkout. This ensures Jenkins can interact with the GitHub repository over standard Git commands during polling and checkout operations.

5. Jenkins Job Creation

A Jenkins Freestyle job named **Community-Health-Survey-CI** was created to implement the Continuous Integration pipeline for the project. The purpose of this job is to automatically retrieve the latest source code from the GitHub repository, execute the Maven build lifecycle (clean, test, package), and archive the generated Spring Boot JAR file whenever a new change is detected on the main branch.

The job configuration is organised into distinct sections — Source Code Management, Build Triggers, Build Steps and Post-build Actions — each of which is documented in the following sections.

6. Source Code Management Configuration

The Jenkins job is configured to use Git as its Source Code Management (SCM) system, pointing to the project's public GitHub repository. Since the repository is public, no Jenkins credentials are required for read access.

- Repository URL: https://github.com/jyolicious/DevOps_miniProject
- Branch Specifier: */main
- Credentials: - none - (public repository)

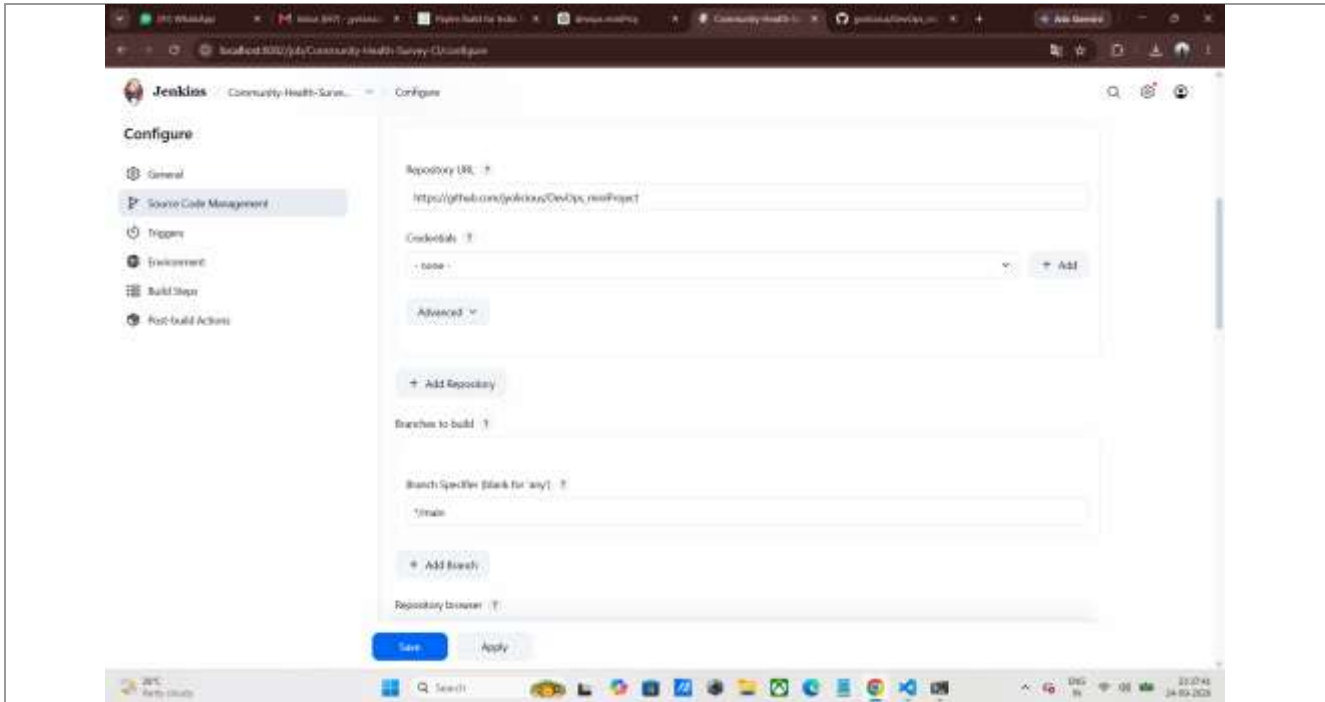


Figure 2: Git SCM configuration connecting the Jenkins job to the GitHub repository.

This configuration ensures that Jenkins checks out the main branch of the repository for every build, which is the branch that carries approved, merged changes for the Community Health Survey System.

7. Poll SCM Configuration

Rather than relying on a GitHub webhook, the job uses the **Poll SCM** trigger, which instructs Jenkins to periodically query the GitHub repository for new commits. The configured schedule is:

```
H/5 * * * *
```

This cron-style expression instructs Jenkins to check the repository roughly every five minutes (the "H" hash symbol spreads the exact minute to avoid load spikes across multiple jobs). If Jenkins detects that the latest commit on the main branch differs from the revision it last built, it automatically schedules a new build.

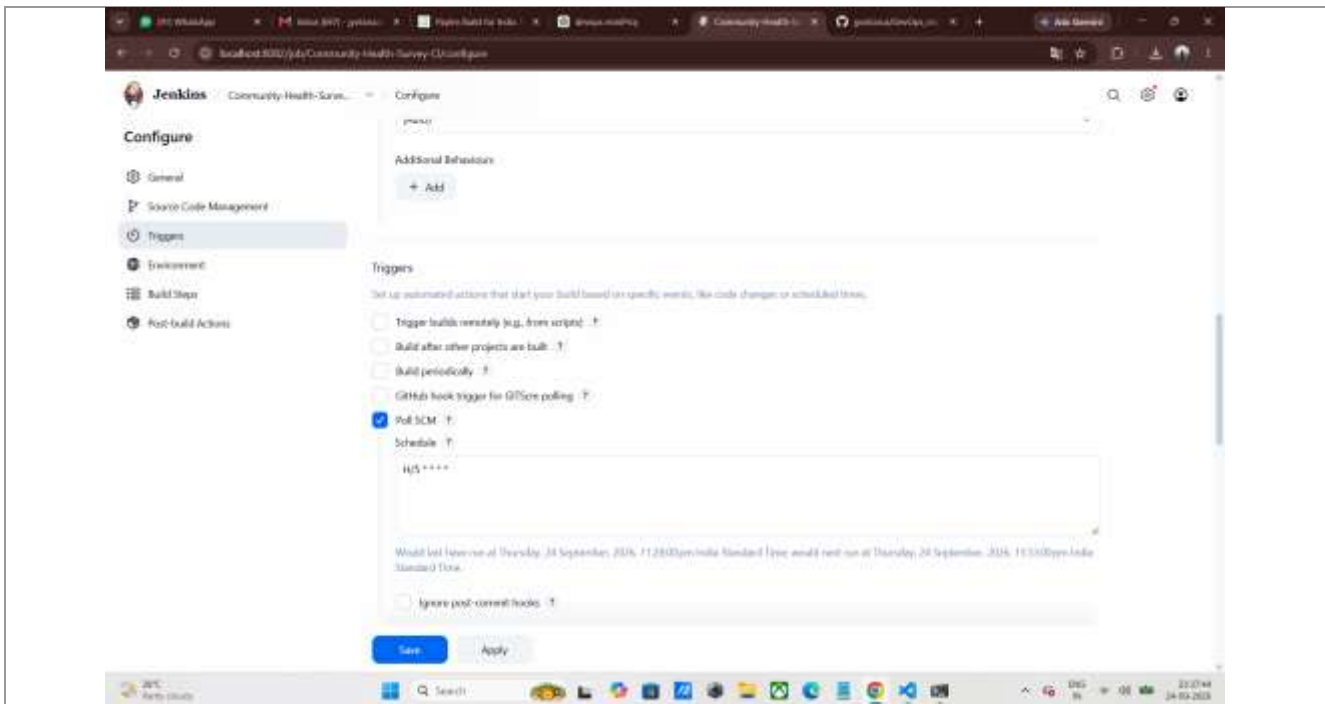


Figure 3: Poll SCM trigger configured to periodically check the GitHub repository.

It should be noted that Poll SCM is a pull-based mechanism initiated from the Jenkins side, and is distinct from a GitHub webhook (push-based). No webhook integration was configured or used for this deliverable.

8. Maven Build Configuration

The Community Health Survey Spring Boot application resides in the `community-health-survey/chss` subdirectory of the repository, while the Maven wrapper script (`mvnw.cmd`) is located at the repository root. To ensure Maven builds the correct project, the Jenkins build step explicitly targets the application's POM file using the `-f` (file) option:

```
call mvnw.cmd -f community-health-survey\chss\pom.xml clean test package
```

The `-f` option is required because the working directory used by Jenkins during the build corresponds to the repository root, where `mvnw.cmd` resides, but the actual Maven project descriptor (`pom.xml`) for the Spring Boot application is nested inside `community-health-survey/chss`. Without explicitly specifying the POM path, Maven would attempt to build from the repository root and fail to locate the correct project. The `clean test package` goals ensure that previous build output is removed, unit tests are executed, and a fresh, tested JAR artifact is produced.

9. Build and Test Process

When triggered, the Jenkins job invokes the Maven wrapper with the command described in Section 8. Maven first cleans any existing target output, then compiles the application source, runs the project's test suite, and finally packages the compiled classes and dependencies into an executable Spring Boot JAR under `community-health-survey/chss/target/`. A successful build is indicated in the Jenkins console output by a `Maven BUILD SUCCESS` message, confirming that compilation, testing and packaging all completed without error.

10. Artifact Archiving

A Post-build Action of type Archive the artifacts was added to the job configuration, using the following file pattern:

```
community-health-survey/chss/target/*.jar
```

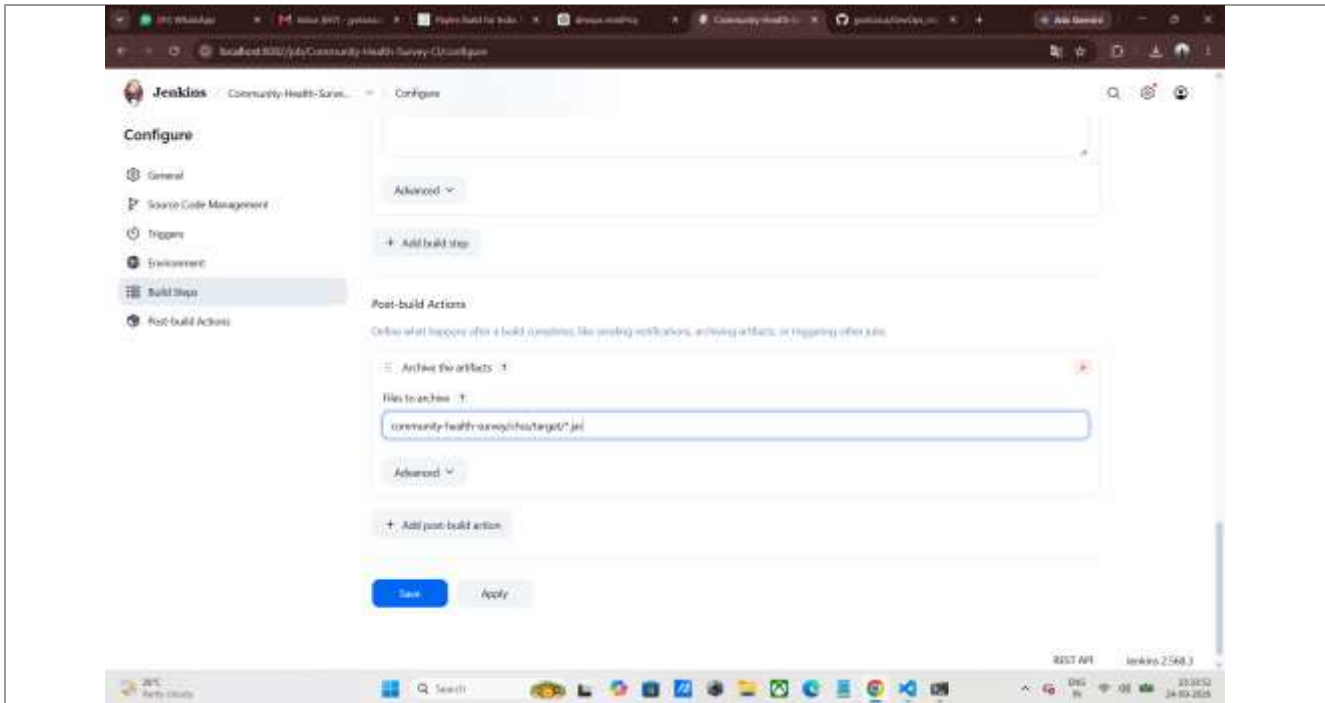


Figure: Post-build "Archive the artifacts" configuration specifying the JAR artifact pattern.

This pattern instructs Jenkins to locate and archive any .jar file produced inside the target directory of the Spring Boot module after a successful build. The archived JAR then becomes available for download directly from the corresponding Jenkins build page, providing a retained, versioned build output for each successful CI run.

11. Continuous Integration Trigger Verification

This section documents the evidence that the Jenkins job was genuinely triggered by an automatic Source Code Management (SCM) change, rather than only through manual invocation.

During the initial setup and debugging of the Jenkins job, several builds were run manually while the Source Code Management, Poll SCM and build step configurations were being finalised. Of these, Build #4 completed successfully and served as confirmation that the job's Maven build and artifact archiving steps were correctly configured.

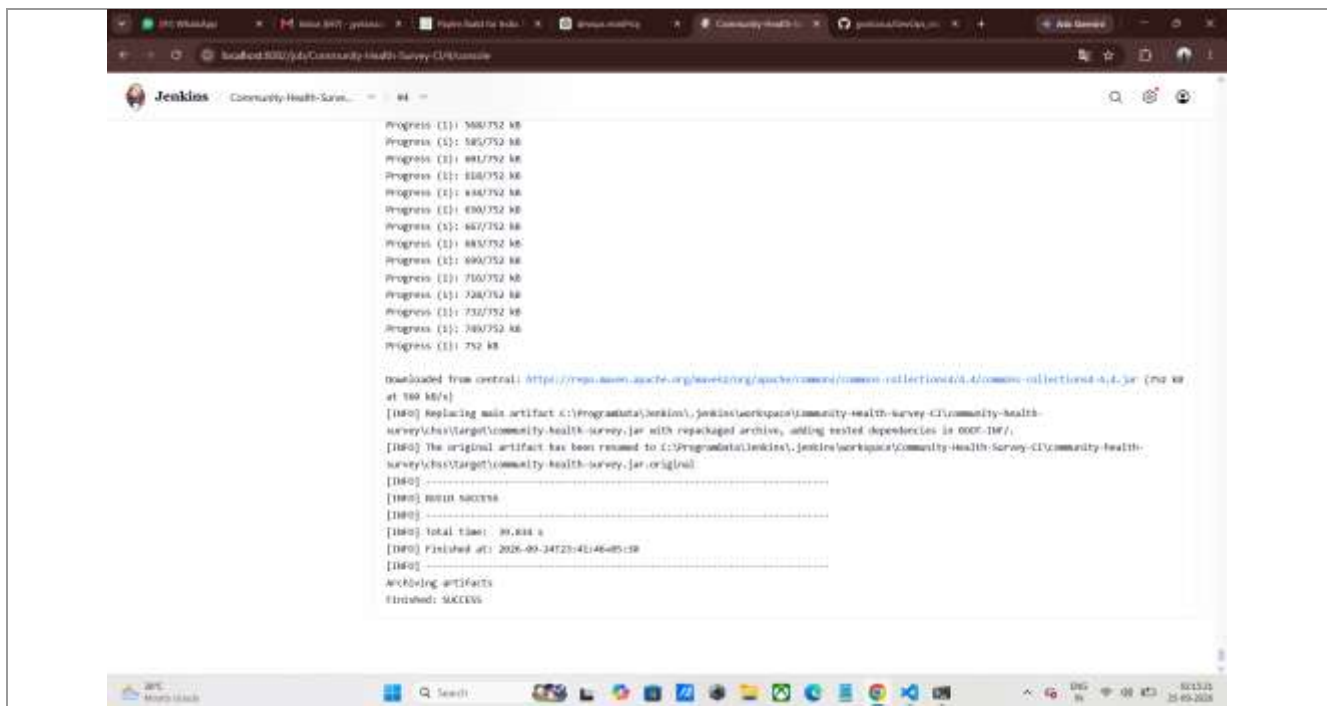


Figure 4: Console output of Build #4, an earlier configuration/verification build, showing Maven execution and BUILD SUCCESS.

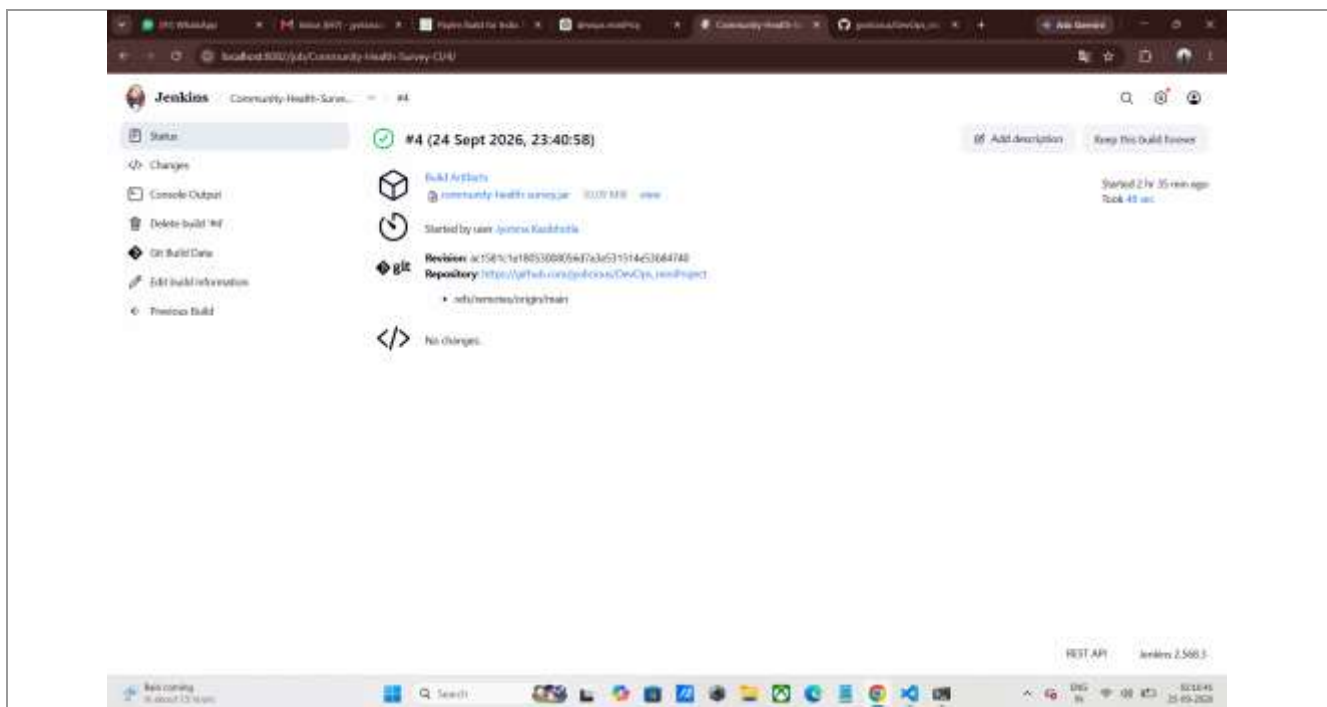


Figure 5: Build #4 status page, showing that the build was started manually by a Jenkins user and archived the community-health-survey.jar artifact.

After the Week 6 report and associated change were committed and pushed to the GitHub repository's main branch, Jenkins' Poll SCM mechanism subsequently queried the repository during its next scheduled poll. The Jenkins Git Polling Log recorded the new remote revision and reported that changes had been found relative to the last built revision:

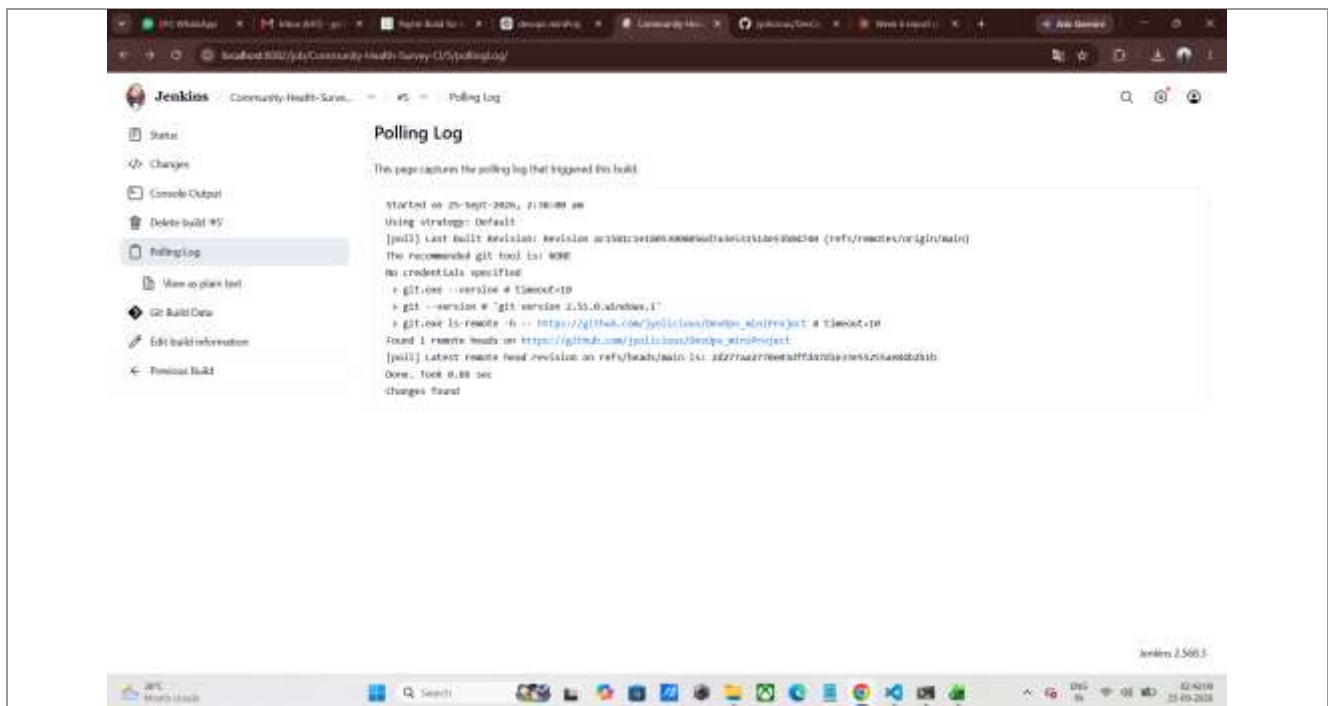


Figure 6: Jenkins polling log for Build #5, showing detection of a new remote revision ("Changes found") relative to the previously built revision.

Because a new revision was detected, Jenkins automatically scheduled and started Build #5. Unlike the earlier builds, the status page for Build #5 explicitly shows "Started by an SCM change" rather than a named user, together with the specific commit ("week 6 report added") that triggered it:

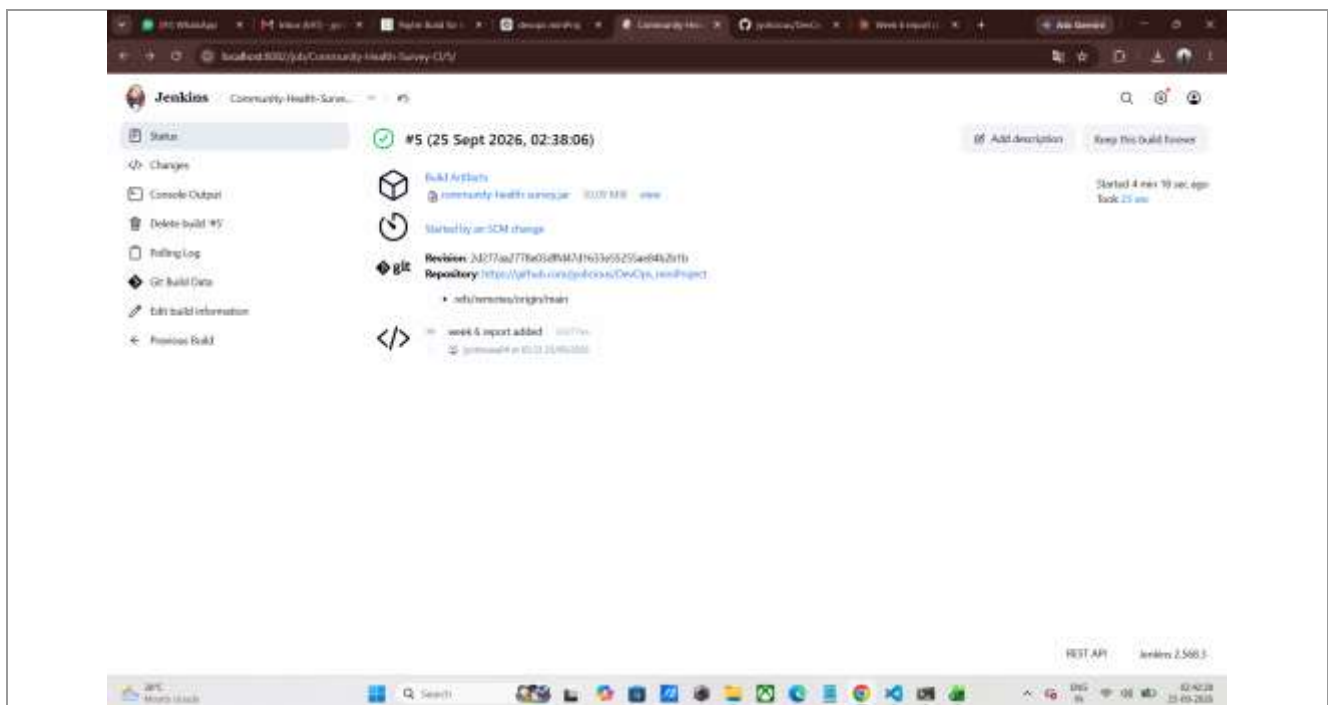


Figure 7: Build #5, automatically triggered through the configured Poll SCM mechanism, showing the triggering commit and the archived JAR artifact.

Build #5 completed successfully and archived community-health-survey.jar (50.09 MiB) as its build artifact. Build #5 is therefore identified as the concrete evidence of SCM-triggered CI execution for this project, distinct from the earlier manually invoked configuration builds such as Build #4.

12. Successful Build Verification

Build #5's status page (Figure 7) confirms a successful completion (green check status), a run duration of 23 seconds, and an archived build artifact of 50.09 MiB. While a dedicated console-output capture of Build #5 was not taken, Build #5 executed the identical Jenkins build configuration and Maven command as Build #4 (Figure 4), whose console output explicitly recorded a Maven BUILD SUCCESS message and successful artifact archiving. Taken together, Figures 4, 6 and 7 provide consistent evidence that the SCM-triggered Build #5 completed the full clean → test → package → archive sequence successfully.

13. Screenshots / Evidence

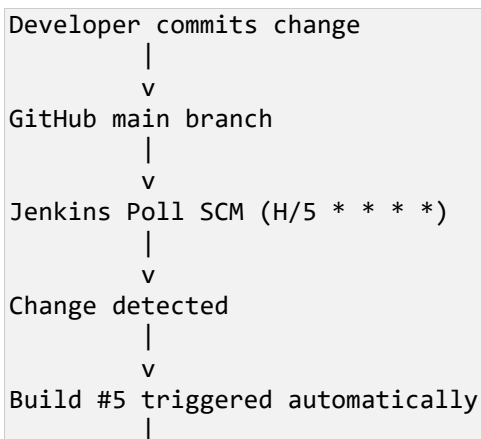
The following figures were captured directly from the running Jenkins instance and are referenced throughout this report:

- Figure 2: Git SCM configuration (repository URL, credentials, */main branch).
- Figure 3: Poll SCM build trigger configuration (schedule H/5 * * * * *).
- Figure 4: Console output of Build #4 (earlier configuration/verification build) showing BUILD SUCCESS.
- Figure 5: Build #4 status page, showing a manually started build and archived artifact.
- Figure 6: Jenkins Git polling log for Build #5, showing detection of a new commit ("Changes found").
- Figure 7: Build #5 status page, showing "Started by an SCM change", the triggering commit, and the archived JAR artifact.

A dedicated screenshot of the GitHub commit page itself was not available and has therefore been omitted in line with the requirement not to invent evidence; the triggering commit ("week 6 report added") is nonetheless visible directly within the Jenkins changelog shown in Figure 7.

14. End-to-End CI Workflow

The following diagram summarises the sequence of events verified in this report, from a developer commit through to an archived build artifact:



```
      v
call mvnw.cmd -f community-health-survey\chss\pom.xml clean test package
      |
      v
Tests / build successful
      |
      v
JAR generated (community-health-survey.jar)
      |
      v
Jenkins archives artifact
```

15. Results

The following results were achieved during Week 7:

- Jenkins 2.568.3 was successfully installed and configured, running locally on <http://localhost:8082> with Java 17.
- The Community-Health-Survey-CI job was created and connected to the GitHub repository [jyolicious/DevOps_miniProject](#) on the main branch.
- Poll SCM was configured with a schedule of H/5 * * * *, enabling periodic, automatic checking of the repository for new commits.
- A genuine SCM change (the Week 6 report commit) was detected by Jenkins' polling mechanism.
- Build #5 was automatically triggered as a direct result of the detected SCM change.
- The Maven build (clean test package) executed successfully against the correct project POM using the -f option.
- The resulting Spring Boot JAR artifact (community-health-survey.jar, 50.09 MiB) was generated and archived by Jenkins.

16. Conclusion

Week 7 of the Community Health Survey System DevOps mini-project successfully established a working Continuous Integration pipeline using Jenkins. A dedicated Jenkins job was configured to connect to the project's GitHub repository, poll it periodically for changes, and execute a Maven-based build that compiles, tests and packages the Spring Boot application. Crucially, the pipeline was validated not only through manually invoked configuration builds (Build #4) but through an automatically triggered build (Build #5) initiated purely by Jenkins' Poll SCM mechanism detecting a genuine commit to the main branch. The successful completion of Build #5 and the archiving of its JAR artifact confirm that the CI requirement for Week 7 — a configured Jenkins job, a successful build log, trigger evidence, and an archived build artefact — has been fully satisfied.

17. Evidence / Acceptance Criteria Table

Requirement	Implementation	Evidence
Jenkins installed and configured	Jenkins 2.568.3 installed and running locally on http://localhost:8082 , with Git and JDK tool installations configured.	Section 4; Manage Jenkins → Tools configuration

GitHub repository connected	Job configured with Git SCM pointing to https://github.com/jyolicious/DevOps_miniProject , no credentials required (public repository).	Figure 2
Main branch configured	Branch Specifier set to */main.	Figure 2
Poll SCM enabled	Poll SCM trigger enabled with schedule H/5 * * * * (approx. every 5 minutes).	Figure 3
Maven build configured	Build step executes call mvnw.cmd -f community-health-survey\chss\pom.xml clean test package.	Section 8
SCM change detected	Jenkins Git Polling Log recorded a new remote revision and reported "Changes found" prior to Build #5.	Figure 6
Automatic Jenkins build triggered	Build #5 shows "Started by an SCM change", distinguishing it from manually started earlier builds.	Figure 7
Successful build	Build #5 completed successfully; Build #4 console output (same pipeline) confirms BUILD SUCCESS.	Figures 4 and 7
Artifact archived	community-health-survey.jar (50.09 MiB) archived and listed under Build Artifacts for Build #5.	Figure 7